

RonaORAEPubSub

Detailed Technical Overview — JSON Transformation Pipeline

1. What the Software Does

RonaORAEPubSub is a **real-time retail transaction translator**. It sits between modern point-of-sale systems (which emit rich, nested JSON events conforming to the **ORAE v2.0.0** standard) and a legacy merchandise management system (MMS) that expects flat, fixed-width records from two tables:

- **RIMSLF** — order / line-item records (items, taxes, fees)
- **RIMTNF** — tender records (payments)

The service subscribes to one Google Cloud Pub/Sub topic, validates and transforms each message, and publishes the result to another topic for downstream warehouse ingestion.

2. Input — `RetailEvent` JSON

File: `PubSubApp/Models/RetailEvent.cs` and `Models/OraeModels.cs`

The incoming JSON is a deeply nested structure. Top-level shape:

```
{
  "schemaVersion": "2.0.0",
  "messageType": "RetailEvent",
  "eventType": "ORIGINAL | CORRECTION | CANCELLATION | SNAPSHOT",
  "eventCategory": "TRANSACTION | TILL | STORE | ...",
  "eventId": "...",
  "occurredAt": "UTC",
  "ingestedAt": "UTC",
  "businessContext": { store, workstation, channel, sequenceNumber },
  "actor": { cashier, customer },
  "transaction": {
    "transactionType": "SALE | RETURN | VOID | POST_VOID | AR_PAYMENT | ...",
    "items": [ TransactionItem, ... ],
    "tenders": [ Tender, ... ],
    "totals": { gross, discounts, tax, net, changeDue, cashRounding }
  },
  "references": { sourceTransactionId },
  "audit": { code, message }
}
```

Key nested models:

- **TransactionItem** — `lineId`, `parentLineId` (for EPP hierarchy), `item` (SKU/GTIN/description), `quantity`, `pricing` (with `priceVehicle` formatted as `"CODE:REF"`), `taxes[]`, `fees[]`, and an

`attributes` dictionary carrying markers like `x-epp-coverage-identifier="9"`.

- **Tender** — `tenderId` ("CA", "VI", "FX", "GC", ...), `amount`, `method`, `authCode`, `emv.tags.magStrip`, last-4.

Deserialization entry point: `RetailEventManager.ReadRecordSetFromString()` (lines 2229–2243) calls `JsonSerializer.Deserialize<RetailEvent>(json)`. `System.Text.Json` is used throughout, with `[JsonPropertyName]` attributes on every model field.

3. Validation — `OraeValidator.ValidateOraeCompliance()`

File: `PubSubApp/Validation/OraeValidator.cs` (lines 11–288). Runs **before** mapping.

- Enforces `schemaVersion == "2.0.0"` and `messageType == "RetailEvent"`.
- Enforces enum membership for `eventType` (4 values) and `eventCategory` (15 values).
- `businessContext.store.currency` must match **ISO-4217** (`^[A-Z]{3}$`).
- Every `Money` value must match `^-?[0-9]{1,18}(\.[0-9]{1,4})?$/`.
- Non-CANCEL/VOID transactions must have ≥ 1 item; every item must have `lineId`, `sku`, `description`, `uom`, `unitPrice`, `extendedPrice`.
- **Ontario-specific rule** (lines 227–257): a single SKU cannot carry both full-HST ($\geq 10\%$) and partial-HST ($< 10\%$) simultaneously.
- **Early exit** for `eventSubType == "transactionBurned"` — only `audit.code/message` required.

If validation fails, the message is **ACKed** (not NACKed) so it doesn't redeliver forever.

4. Transformation — `RetailEventManager.MapRetailEventToRecordSet()`

File: `PubSubApp/RetailEventManager.cs` (~2,245 lines; main method 11–1460). This is the heart of the system.

4a. Pre-flight decisions (lines 20–67)

- Scan items for **employee discount** (`priceVehicle` contains `"EMP"`).
- Scan tenders for **gift card**.
- Check for **customerId**.
- These three flags feed `MapTransTypeSLFTTP()` and `MapTransTypeSLFLNT()` — a decision tree mapping `transactionType` + flags to legacy 2-digit type codes (SALE=01, EMP SALE=04, RETURN=11, VOID=87, POST_VOID=88, AR_PAYMENT=43, SALE+GC=45...).
- `ApplyTimezoneAdjustment()` — prefers IANA (`America/Toronto`) from `store.timeZone` , falls back to a store-ID heuristic (BC/AB = -8h, ON/QC = -5h); produces local `YYMMDD` / `HHMMSS` integers.
- **SalesPerson** (SLFSPS): SCO registers (ID starts with "8") use register ID; ACO uses `cashier.loginId` .

4b. `transactionBurned` short-circuit (lines 1593–1635)

Returns a `RecordSet` with **one** `TenderRecord` (TNFTTP="99", FundCode="CA", zero amount). No `OrderRecords`.

4c. Item → `OrderRecord` loop (lines 84–493)

Per item:

- **Quantity:** `quantity.value × 100` → integer cents; sign is "-" for RETURN.
- **Prices:** check both new (`pricing.priceOverride.overrideUnitPrice`) and legacy (`pricing.override`) override sources. SLFSEL uses override if non-zero, else unitPrice. SLFEXT = qty × effective price.
- **Price Vehicle:** split `priceVehicle` on ":" → SLFPVC (4 chars) / SLFREF (12 chars). If employee discount without priceVehicle, force `SLFPVC="EMP"` .
- **Ad code:** SLFADC = "#####" if unitPrice ≠ originalUnitPrice, else "0000".
- **EPP marker:** SLFZIP = 9 spaces + `x-epp-coverage-identifier` digit. EPP children get SLFTTP/SLFLNT = "21".
- **Reason code** (SLFRSN): "RRT0" for RETURN, "VOD0" for VOID, "POV0"+reason for override, "IDS0"+reason for manager override.
- **Eco fees** (lines 391–492): each fee with amount > 0 becomes an additional `OrderRecord` with `LineType="83"` .

4d. EPP reordering (lines 495–612)

After the first pass, records are reordered so each EPP child (items with `x-epp-coverage-identifier="9"`) appears **immediately after its parent** (matched by `parentLineId`). Eco fees travel with the parent block. Uses `itemRangeStart` / `itemRangeCount` tracking plus an `eppChildMap` keyed by parent lineId.

4e. Tax consolidation (lines 614–1332)

The most intricate business logic. Rather than emit one tax record per item-tax, the mapper **consolidates taxes per province**:

Province	Record(s) Emitted
Ontario	HON (XH, HST $\geq 10\%$) + HON1 (XI, GST $< 10\%$)
BC / MB / SK / QC	FED (XG, GST) + Provincial (XR / XM / XS / XQ)
Alberta	FED only (XG)
Other	One record per item tax authority

Tax fields (e.g., SLFACD="HON", SLFTCD=taxCode, SLFEXT/SLFSEL = aggregated amounts in cents, 11/9 digits). OriginalTx fields (SLFOTS/SLFOTD/SLFOTR/SLFOTT) are zeroed for SALE, populated for RETURN/VOID.

4f. Sequence numbering (lines 1334–1350)

All item records first, then tax records, then tenders — each gets a 5-digit zero-padded `TransSeq` starting at 1.

4g. Tender mapping (lines 1352–1449)

Per tender in `transaction.tenders`:

- Base record with TNFTDT/TNFTTM from `transactionDateTime`, TNFCLK/TNFREG from register, TNFTTX from sequence.
- TNFFCD = `tender.tenderId` ("CA" / "VI" / "FX" / "GC" / ...).
- Amount = 11-digit cents; sign stored separately.
- For card tenders: TNFCCD = last-4 padded with '*' to 19 chars; TNFAUT = authCode (last 6 digits for Flexiti "FX", first 6 for others); TNFMSR = mag-strip flag.

CASH transactions spawn up to 3 records:

- Main CASH payment
- **CA / change-due** record (sign always "-")
- **PR / penny rounding** record (sign inverted from ORAE)

Fallback: if `tenders[]` is empty, synthesize one CA tender for the net total.

5. Output — `RecordSet` JSON

File: `PubSubApp/Models/RecordSet.cs`

The output shape is intentionally flat and schema-like:

```
{
  "RIMSLF": [ OrderRecord, OrderRecord, ... ],
  "RIMTNF": [ TenderRecord, TenderRecord, ... ]
}
```

- **OrderRecord** — ~80 fields mirroring the MMS RIMSLF table (SLFTTP, SLFLNT, SLFTDT, SLFTTM, SLFSKU, SLFQTY, SLFORG, SLFSEL, SLFEXT, SLFDSA, SLFTX1–4, SLFRSN, ASFPRO / SST / IST, ...). `SourceLineId` / `SourceParentLineId` are `[JsonIgnore]` 'd — only used during EPP reordering.
- **TenderRecord** — ~40 fields mirroring RIMTNF (TNFTTP, TNFFCD, TNFAMT, TNFCCD, TNFEXP, TNFAUT, TNFMBR, TNFZIP, TNFEML, ATFSST / IST / ORD / PRO, ...).

Formatting discipline: every currency field is 11-digit zero-padded cents. Every string is exact-width (space- or zero-padded). `[StringLength]` and `[Range]` attributes on the models enforce the contract; null lists are omitted via `JsonIgnoreCondition.WhenWritingNull`.

Publishing (`PubSubApp/Program.cs` ~line 500)

The outgoing Pub/Sub message:

- **Body** = `JsonSerializer.Serialize(recordSet)`
- **Attributes** = input attributes forwarded verbatim + response metadata (original messageId, processing status, timestamps, etc.)

6. Core Design Principles of the Converter

#	Principle	Description
1	Units	Decimals become integer cents ($\times 100$) stored as zero-padded strings.
2	Consolidation	Item-level taxes collapse into province-specific aggregate records, not a 1:1 passthrough.
3	Hierarchy preservation	EPP items reorder under their parent line, respecting <code>parentLineId</code> .
4	Tender expansion	CASH tenders can fan out into main + change + rounding records.
5	Timezone fidelity	Store-local dates/times (IANA-aware) rather than UTC.
6	Fixed-width contract	Every field is padded/truncated to exact MMS-schema widths, enforced by model attributes.
7	Fail-safe ACKing	Semantic failures (bad JSON, failed validation) ACK to prevent infinite redelivery; only transient/infrastructure failures NACK.

Generated 2026-04-21 from the `kind-cray` worktree.